

Mohsin Ibna Hossain

#Final
Term

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Table of Contents

SL.NO	Topic	Pages
1	2D Array	03-09
2	String	10-21
3	Pointer	22
4	Dynamic Memory Allocation	23-24
5	Function	24-37
6	Function Overloading	37-40
7	Local Variable	41-42
8	Global Variable	43-44
9	Call by Value	45-46
10	Call by Reference	47-48
11	Structure	49-56
12	Access Modifier/Specifier	57-62
13	Constructor & Destructors	63-67
14	Inheritance	68-74
15	Static Variable	75-78

#2D Array

	0	1	2
0			
1			
	0	1	2

```
int arr[3][3];
```

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{  
    int arr[2][3] = {5, 10, 15, 20, 25, 30};
```

```
    cout << arr[1][2];
```

Output:

30

	<u>2/20</u>		
	0	1	2
0	5	10	15
1	20	25	30
	0	1	2


```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
    int arr[3][3] = {5, 10, 15, 20, 25, 30, 35, 40, 45};
    cout << arr[2][1];
}
```

Output:-

40

5	10	15
20	25	30
35	40	45

How to use loop in 2D array:

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
    int arr[3][3] = {5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 60};
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Output:-

```
5 10 15
20 25 30
35 40 45
```

Tracing

i = 0, j = 0 → j = 5	i = 1, j = 2 → j = 40
j = 1 → j = 10	i = 2, j = 0 → j = 50
j = 2 → j = 15	j = 1 → j = 60
i = 1, j = 0 → j = 20	
j = 1 → j = 25	

What if want to
take input from user?

```
# include <iostream>
```

```
using namespace std;
```

```
int main () {
```

```
    int arr[3][3];
```

```
    for (int i=0; i<3; i++) {
```

```
        for (int j=0; j<3; j++) {
```

```
            cin >> arr[i][j];
```

```
        }
```

```
    }
```

```
    for (int i=0; i<3; i++) {
```

```
        for (int j=0; j<3; j++) {
```

```
            cout << arr[i][j] << " ";
```

```
        }
```

```
    }
```

```
    cout << endl;
```

```
    return 0;
```

```
}
```


How to take input for 2D Array

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < m; j++)  
        cin >> arr[i][j];
```

How to print output for 2D Array

```
* for (int i = 0; i < n; i++)  
    for (int j = 0; j < m; j++)  
        cout << arr[i][j] << " ";  
    cout << endl;
```

take 2D Array & take input from user
& DO Sum and Average

```
#include <iostream>  
using namespace std;  
int main() {  
    int arr[2][4];  
    int sum = 0;
```



```
float avg = 0;
cout << "Enter Your 2D Array Elements";
```

```
for (int i = 0; i < 2; i++)
```

```
for (int j = 0; j < 4; j++)
```

```
cin >> arr[i][j];
```

```
sum = sum + arr[i][j];
```

```
avg = (float) sum / 8;
```

```
cout << "Total sum" << sum << endl;
```

```
cout << "Average" << avg << endl;
```

```
return 0;
```

Matrix Addition:

$$= \begin{bmatrix} 2 & 0 & 1 \\ 4 & 2 & 0 \\ 0 & 1 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 0 \\ 2 & 2 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 2+1 & 0+1 & 1+1 \\ 4+0 & 2+1 & 0+0 \\ 0+2 & 1+2 & 1+2 \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 1 & 2 \\ 4 & 3 & 0 \\ 2 & 3 & 3 \end{bmatrix}$$

Take 2 Matrix 3x3 and store them in
separate Array and Do Sum the matrix
And Print the Sum.

#include <iostream>

using namespace std;

int main()

{
int m[3][3];

cout << "Enter your 3x3 Matrix Elements:";

for (int i=0; i<3; i++)

{
for (int j=0; j<3; j++)

{
cin >> m[i][j];

}

cout << "Enter your next 3x3 Matrix Elements:";

int n[3][3];

for (int i=0; i<3; i++)

{
for (int j=0; j<3; j++)

{
cin >> n[i][j];

}

int sum[3][3];

for (int i=0; i<3; i++)

{
for (int j=0; j<3; j++)

{
sum[i][j] = m[i][j] + n[i][j];

cout << "Sum of the matrix:" << endl;

cout << sum[i][j] << " ";

return 0;

V.V.G for output tracing / 100% - 2TH (2)

#Strings

- ① strlen() → String length
- ② Strcat() → String concatenate
- ③ Strncpy() → String copy
- ④ Strncat() → String copy n characters
- ⑤ Strncat() → String concatenate n characters
- ⑥ Strncmp() → String compare
- ⑦ Strncmp() → String compare n characters.
- ⑧ Strchr() → String character.
- ⑨ Strstr() → String String

⇒ ~~অন্যতম~~ build function. ~~এখানে~~ ~~এখানে~~ ~~এখানে~~ ~~এখানে~~
~~এটি~~ heading file ~~এখানে~~ ~~এখানে~~ ~~এখানে~~ ~~এখানে~~
include <string>.

String length - (strlen) { continues till

⇒ ~~if array contains '\0' then string length measured~~
~~if array contains '\0' then string length measured~~

① #include <iostream>
 #include <cstring>
 using namespace std;
 int main()

char s1[15] = "ABC";
 cout << "strlen(s1) = " << strlen(s1) << endl;

Output: strlen(s1) = 3

② int main()

char s1[15] = "A B C";
 cout << "strlen(s1) = " << strlen(s1) << endl;

Output: strlen(s1) = 5

* termination char = '\0' ~~array contains '\0' then string length measured~~
 char count 24-11.


```
* int main() { (10) printf("#
```

```
char s1[15] = "ABC\0";
```

```
cout << "strlen(s1):-" << strlen(s1) << endl;
```

```
}
```

Output:- strlen(s1):- 3

```
* int main() {
```

```
char s1[15] = "\0ABC";
```

```
cout << strlen(s1) << endl;
```

```
}
```

Output:- 0

```
* int main() {
```

```
char s1[15] = "AB\0C";
```

```
cout << "strlen(s1):-" << strlen(s1) << endl; }
```

Output:- strlen(s1):- 2

String Concatenate (Strcat) :-

→ এই ফাংশনটি ২-২০ এর String-এর সাথে
নতুন String যুক্ত করে দেয়।

```
#include <iostream>
#include <cstring>
using namespace std;
int main()
{
    char s1[15] = "ABC";
    char s2[15] = "DE";
    cout << "Strcat(s1,s2) << strcat(s1,s2) << endl;
    return 0; }
```

Output :- Strcat(s1,s2) ABCDE

```
int main()
{
    char s1[15] = "ABC"
    char s2[15] = "DEF"
    cout << "s1" << s1 << endl;
    cout << "Strcat(s1,s2) << strcat(s1,s2) << endl;
    cout << "s1" << s1 << endl; }
```

Output :-
s1 ABC
Strcat(s1,s2) ABCDEF
s1 ABCDEF

Notes - String Array નો loop દ્વારા

Array print કરાયા, for loop દ્વારા
Array input લેવામાં આવે.

Notes 2 :- String એ Array જેવો છે પણ એમાં
જ્યાં String નો value change થાય ત્યાં જ્યાં String
નો value ફેરવવામાં આવે ત્યાં જ્યાં String
નો value ફેરવવામાં આવે ત્યાં જ્યાં String

"ABC" = [0] 0 1 2

"ED" = [0] 1 2

String concatenate n char

Strncat

→ જો String નો value જોડવા/મિલવવા
કરે તો String નો value જોડવામાં આવે.

Examples -

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main()
```

```
{
    char s1[15] = "ABC";
```

```
    char s2[15] = "DEF";
```

```
    cout << s2 << endl;
```

```
    cout << "Strncat(s2, s1, 1) << Strncat(s2, s1, 1) << endl;
```

```
    cout << s2 << endl; }
```


Output—

S2% DEP

Strncat(S2, S1, 1)%-DEPA

S2% DEPA

String Copy: (strcpy%)

⇒ strcpy(S1, S2) — এখানে S1-এ S2-এর কপি করা হয়।
যদি S1-এর S2-এর কপি করার জন্য পর্যাপ্ত স্পেস না থাকে তবে সেগমেন্টেশন ফল্ট হবে।

Examples—

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main ()
```

```
{ char s1[15] = "ABC";
```

```
  char s2[15] = "DEP";
```

```
  cout << "s2%" << s2 << endl;
```

```
  cout << "strcpy(s2, s1)% " << strcpy(s2, s1) << endl;
```

```
  cout << "s2%" << s2 << endl; }
```

Output—

S2% DEP

strcpy(s2, s1)% ABC

S2% ABC


```
* int main() {
```

```
    char s1[15] = "AB \0C";
```

```
    char s2[15] = "DEPW";
```

```
    cout << "s2" << s2 << endl;
```

```
    cout << "strcpy(s2, s1)" << strcpy(s2, s1) << endl;
```

```
    cout << "s2" << endl; }
```

Output:-

s2: DEPW

~~DEPW~~

strcpy(s2, s1): AB

s2: AB

String copy n char: (strcpy):

⇒ ଏହି ଫଙ୍କ୍ସନ୍ ଦେଇ ଏକ ସ୍ଟ୍ରିଙ୍ଗ କପି କରାଯାଇଥାଏ
ତା n-ଟି Value ବାରିଏବଲ୍ ମଧ୍ୟରେ କରାଯାଇଥାଏ ।
ଏହା ଏହି ଫଙ୍କ୍ସନ୍ ଦେଇ ସ୍ଟ୍ରିଙ୍ଗ କପି କରାଯାଇଥାଏ ତା n-ଟି ବାରିଏବଲ୍
Value ମଧ୍ୟରେ କପି କରାଯାଇଥାଏ ଏବଂ ଏହା ଏକ modify
କରି ନଥାଏ ।

Example:-

P. T. O

#include <iostream>

#include <string>

using namespace std;
int main()

char s1[15] = "ABC";

char s2[15] = "DFPW";

cout << "s2" << s2 << endl;

cout << "strncpy(s2, s1, 3) %>" << strncpy(s2, s1, 3) << endl;

cout << "s2" << s2 << endl;

Output:

s2 DFPW

strncpy(s2, s1, 3) %> ABCW

s2 %> ABCW

String Compare (strcmp):

⇒ strcmp string ko compare karati hai.

strcmp(s1, s2) - agar s1 aur s2 ki har har ek char ka

ASCII Value same ho to 0, agar koi char ka ASCII

Value different ho to koi number return karta hai.

if $s_1 > s_2 \Rightarrow 1$
 $s_1 = s_2 \Rightarrow 0$
 $s_1 < s_2 \Rightarrow -1$

the condition of
Comparison.

* Example -

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    char s1[15] = "A";
    char s2[15] = "B";
    cout << "strcmp(s2, s1) %< strcmp(s2, s1) << endl;
```

Output:- strcmp(s2, s1) % 1

String Compare n char

strcmp

⇒ Value compare karne ke liye
n no. value dene hain.

Example:-

```
#include <iostream>
#include <string>
using namespace std;
int main() {
```

```
    char s1[15] = "ABCD";
```

```
    char s2[15] = "ABEF";
```

```
    cout << "strcmp(s2, s1, 3) %< strcmp(s2, s1, 3) << endl;
```

Output:- strcmp(s2, s1, 3) % 1

String Characters (Streh n):

⇒ char variable stores single character
 - char is a primitive data type
 - char is a single character
 - char is a single character
 - char is a single character

Example:—

```
#include <iostream>
#include <string>
using namespace std;
int main () {
```

Char: s1[15] = "ABeD";

char s2 = 'B';

```
Gout << "Streichholz" << streichen(s1, s2) << endl;
```

output - Strchar Bed

```
* int main() {
```

```
char s[16] = "Hello world";
```

```
char s2 = 'l';
```

cout << "strehz(s1, s2): " << strehz(s1, s2) << endl;

Output: $\text{Strech}(s_1, s_2) = \text{No word}$

String String (strstr) :-

→ ଏହା string ଓ string ଖୋଜିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ ।
- ଯଦି ଏହା string ଖୋଜିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ ତେବେ ଏହା
- ଏହା string ଖୋଜିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ ।
Print ହେଉଛି ଏହା ଖୋଜିବା ପାଇଁ ବ୍ୟବହୃତ ହୁଏ ।
null print ହେଉଛି

Example -

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    char s1[16] = "ABCD";
    char s2[16] = "BC";
    cout << "strstr(s1,s2) : " << strstr(s1,s2) << endl;
}
```

output strstr(s1,s2) : BC

```
* int main()
{
    char s1[16] = "ABCD";
    char s2[16] = "BD";
    cout << "strstr(s1,s2) : " << strstr(s1,s2) << endl;
}
```

output : null

String Variable:

⇒ String Variable is a type of Variable that stores a sequence of characters, such as letters, numbers, symbols, and spaces. In C++, a String Variable is declared by using a "string" keyword.

Ex:-

```
String myStringVariable = "Hello, World";
```

```
#include <iostream>
using namespace std;
int main() {
```

```
    String Var1[3] = {"Apple", "Grape", "Orange"};
    for (int i = 0; i < 3; i++) {
        cout << Var1[i] << " ";
    }
    return 0;
```

Output:- Apple, Grape, Orange.

Address findout

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int a = 5;
```

```
    cout << "Address of a:" << &a << endl;
```

```
    float b = 10.5;
```

```
    cout << "Address of b:" << &b << endl;
```

```
    return 0;
```

Output:-

Address of a:
Address of b:

Note - Address no value of variable

Pointers

```
#include <iostream>
using namespace std;
int main()
{
    int a = 50;
    int *ptr = &a;

    cout << "Address of the pointer" << ptr << endl;
    cout << "Address of a" << &a << endl;
    cout << "Address of the stored variable" << ptr << endl;
    cout << "Value of the stored variable" << *ptr << endl;
    return 0;
}
```

Output:-

Address of the pointer: 0x61fe10
Address of a: 0x61fe10
Address of the stored variable: 0x61fe10
Value of stored variable: 50

Notes:- Address of a and Address of the stored variable are same. Value of a and Value of the stored variable are same.

→ Pointer - It stores the address of the variable. It is used to store the address of the variable. It is used to store the address of the variable.

Print - It is used to print the value of the variable. It is used to print the value of the variable.

cout << "Enter the address:" << endl;

Dynamic Memory Allocation

→ এটি হলো প্রোগ্রাম চলার সময় মেমোরি দখল
আয়োজনের দক্ষতা, যা সাহায্যে প্রোগ্রাম খুব দ্রুত
Execution এর সময় মেমোরি দখল দ্রুত হয়
বাড়ানোর আশঙ্কা করে নেই।

Examples:-

```
#include <iostream>
using namespace std;
int main()
{
    int input;
    string reply;
    int *ptr;
    int element;
    cout << "Enter an element:";
    cin >> input;
    cout << "Do you want to add more elements?";
    cin >> reply;
    if (reply == "yes" || "YES")
    {
        cout << "How many Element do you want to add?";
        cin >> element;
        ptr = new int[element + 1] // The system of memory allocates
        ptr[0] = input;
        cout << "Enter the element:";
```



```
for (int i = 1; i < element + 1; i++) {
    cin >> arr[i];
}
```

cout << "Inputed element are:";

```
for (int i = 0; i < element + 1; i++) {
    cout << arr[i] << " ";
}
```

delete[] arr; // memory deallocation.

```
else { cout << "Inputed element is:" << input << endl; }
```

return 0;

Input: 1, 2, 3, 4
Output: 1 2 3 4

Function:

Void function:

⇒ A function is a block of code that performs a specific task and returns a value.

- A function is a block of code that performs a specific task and returns a value.

1. Parameterless

Ex- Void addition () { }

2. Parameterized

Ex- Void addition (int a, int b) { }

⇒ A function which takes no parameter is called a parameterless function.

⇒ A function which takes one or more parameters is called a parameterized function.

Notes - A function which returns a value is called a function with return type. A function which does not return a value is called a void function. A function which is called from the main function is called a called function.

Write a C++ program using the Void function and using that function to add two integers.

```
#include <iostream>
using namespace std;
```

```
void Addition(int a, int b)
```

```
{
    int sum;
```

```
    sum = a + b;
```

```
    cout << "Sum: " << sum << endl;
```

```
int main()
```

```
{
    Addition(10, 50);
```

```
    Addition(100, 30);
```

```
    Addition(2, 50);
}
```


Output:-
Sum: 60
Sum: 100
Sum: 52

Write for three integer Value:

```
int #include <iostream>  
using namespace std;
```

```
void Addition(int a, int b, int c)
```

```
{  
    int Sum;
```

```
    Sum = a+b+c;
```

```
    cout << "Sum: " << sum << endl;
```

```
}  
int main()
```

```
{  
    Addition(40, 20, 60);
```

```
    Addition(30, 20, 10);  
}
```

Output:-
Sum: 120
Sum: 60

If user want to input two Value for Sum:-

```
#include <iostream>
```

```
using namespace std;
```

```
void Add(int a, int b)
```

```
{  
    int sum = a+b;
```

```
    cout << "Sum: " << sum << endl;
```



```

int main() {
    int m, y;
    cout << "Enter two Numbers:";
    cin >> m >> y;
    Add(m, y);
}

```

Outputs-

Enter two numbers: 30, 35
Sum: 65.

Write a C++ program using the Void function.
And using that function to add, subtract, multiply
and divide 2 integers. take value from user.

```

#include <iostream>
using namespace std;

Void add(int a, int b) {
    int sum = a + b;
    cout << "Sum" << sum << endl;
}

Void sub(int a, int b) {
    int sub = a - b;
    cout << "Sub" << sub << endl;
}

```



```

Void mult (int a, int b) {
    int mult = a * b;
    cout << "mult: " << mult << endl;
}

```

```

Void div (int a, int b) {
    int div = a / b;
    cout << "Div: " << div << endl;
}

```

```

int main () {
    int x, y;
    cout << "Enter two integer: ";
    cin >> x >> y;
    add (x, y);
    sub (x, y);
    mult (x, y);
    div (x, y);
}

```

Output: — Enter two integer: 30, 15

Sum: 45

Sub: 15

Mult: 450

Div: 2

write a c++ program using void
function take a value from user &
check this value is even or odd number:-

```
#include <iostream>
using namespace std;
```

```
void evenodd(int a) {
    if (a % 2 != 0) {
```

```
        cout << "It's a odd number";
```

```
    } else { cout << "It's a even number";
```

```
int main() {
```

```
    int n;
```

```
    cout << "Enter a number:";
```

```
    cin >> n;
```

```
    evenodd(n);
```

□

output:- Enter a number: 5
It's a odd number

Write a c++ Program. Using the Void function
take a Value from user & check this Value even or
odd. In 3 times:

```
#include <iostream>
using namespace std;
Void evenodd () {
    int a;
    cout << "Enter a number:";
    cin >> a;
    if (a % 2 == 0) {
        cout << "a is a odd number" << endl;
    }
    else { cout << "a is a even number" << endl; }
}

int main() {
    evenodd();
    evenodd();
    evenodd();
}
```

Output:- Enter a number: 5
5 its a odd number
Enter a number: 10
10 its a even number.
Enter a number: 15
15 its a odd number.

06-04-2023
m o m

Integer type functions

```
#include <iostream>
```

```
using namespace std;
```

```
int addition (int a, int b) {
```

```
    int sum = a + b;
```

```
    return sum; }
```

```
int main() {
```

```
    addition(5, 4);
```

```
}
```

Output:-
mom

আমরা কোন output পাবো না, কারণ

আমরা return cout দিচ্ছি না

Note:-

যদিও এটি integer function এ return type দেওয়া হয়েছে

কিন্তু এটি return type function ও বলা যায়।

```
#include <iostream>
```

```
using namespace std;
```

```
int addition (int a, int b) {
```

```
    int sum = a + b;
```

```
    return sum; }
```

```
int main() {
```

```
    cout << "Sum:" << addition(5, 6); }
```

```
}
```

Output:-
mom

Sum: 11

[illegible]

Root functions —

```
#include <iostream>
using namespace std;
float add (float a, float b) {
    float sum = a + b;
    return sum;
}
```

```
int main() {
    cout << "Float sum:" << add(4.5, 5.5) << endl;
}
```

Output — Float sum 10.0

Character function:-

```
#include <iostream>
```

```
using namespace std;
```

```
char print(char var1){
```

```
    return var1;
```

```
{  
int main(){
```

```
    cout << "print character:" << print('m') << endl;  
}
```

Output:-
print character: m

```
#include <iostream>
```

```
using namespace std;
```

```
char print(char var1){
```

```
    return
```

```
    '0';
```

```
{  
int main(){
```

```
    cout << "print character:" << print('0') << endl;  
}
```

Output:-
print character: 0

Write a c++ program use Void function
and array to check the number is even
or odd

```
#include <iostream>
```

```
using namespace std;
```

```
Void evenOdd (int arr[4])  
{  
    for (int i=0; i<4; i++)  
    {  
        if (arr[i] % 2 == 0)  
            cout << arr[i] << " is a even number" << endl;  
        else  
            cout << arr[i] << " is a odd number" << endl;  
    }  
}
```

```
int main ()
```

```
{  
    int arr[4] = {2, 5, 7, 8};
```

```
    evenOdd(arr);  
}
```

Output —
from 2 is a even number.
5 is a odd number.
7 is a odd number.
8 is a even number.

What if you want to take input
from user?

```
#include <iostream>
using namespace std;
void evenodd(int arr[4])
{
    for(int i=0; i<4; i++)
    {
        if (arr[i] % 2 == 0)
        {
            cout << arr[i] << " is a odd number" << endl;
        }
        else
        {
            cout << arr[i] << " is a even number" << endl;
        }
    }
}

int main()
```

```

{
    int array[4];
    cout << "Enter Numbers";
    for(int i=0; i<4; i++)
    {
        cin >> array[i];
        evenodd(array);
    }
}
```

Output— Enter Numbers 1, 5, 6, 8
1 is a odd number.
5 is a odd number.

6 is a even number.

8 is a even number.

#Write a c++ program to check the inputed Value is Vowel or Consonant use Void function and array.

```
#include <iostream>
```

```
using namespace std;
```

```
void vowelconsonant(char a[5]) {
```

```
    for(int i=0; i<4; i++) {
```

```
        if(a[i] == 'A' || a[i] == 'E' || a[i] == 'I' ||  
            a[i] == 'O' || a[i] == 'U') {
```

```
            cout << a[i] << "Vowel" << endl; }
```

```
        else { cout << a[i] << "Consonant" << endl; } }
```

```
int main() {
```

```
    int arr[5];
```

```
    cout << "Enter 4 character";
```

```
    for(int i=0; i<4; i++) {
```

```
        cin >> arr[i]; }
```

```
    vowelconsonant(arr);
```


Output:- Enter 4 character: A, B, O, C, E

A Vowel

B Consonant

O Vowel

C Consonant

Function Overloading: ~~*****~~

⇒ Same Name ~~is~~ Multiple function ~~in~~ ~~one~~ code ~~is~~ ~~not~~ ~~possible~~ ~~for~~ ~~multiple~~ ~~function~~ ~~overloading~~ ~~in~~ ~~C++~~.
~~for~~ ~~multiple~~ ~~function~~ ~~overloading~~ ~~in~~ ~~C++~~ ~~is~~ ~~not~~ ~~possible~~.

Conditions:-

1. Number of parameters different ~~is~~ ~~not~~ ~~possible~~.

2. Parameters Same ~~is~~ ~~not~~ ~~possible~~ Variable ~~is~~ ~~not~~ ~~possible~~ different ~~is~~ ~~not~~ ~~possible~~.

~~is~~ ~~not~~ ~~possible~~ Error ~~is~~ ~~not~~ ~~possible~~.

Q: Write a C++ Program for
function Overloading.

```
#include <iostream>
```

```
using namespace std;
```

```
void add(int a, int b)
```

```
{  
    int sum = a + b;
```

```
    cout << "Sum:" << sum << endl;  
}
```

```
void add(int a, int b, int c)
```

```
{  
    int sum = a + b + c;
```

```
    cout << "Sum:" << sum << endl;  
}
```

```
void add();
```

```
{  
    cout << "parameterless" << endl;  
}
```

```
void add(string Var1, int a)
```

```
{  
    cout << "String:" << Var1 << endl;  
}
```

```
int main()
```

```
{  
    add(1, 5, 15);
```

```
    add("Hello", 5);
```

```
    add();
```

```
    add(10, 20);  
}
```

Output:-

Sum: 23

String: Hello

parameterless.

Sum: 30.

#When compiler gives us the Ambiguous problem:—

⇒ When function overloading is done with parameters as integer type variable and floating type variable then compiler gives us an ambiguous problem because:—

because:—

1. When we pass integer type variable to a floating type value then the integer portion is truncated.

2. When we pass floating type variable to an integer type value then the floating type value is truncated and Accepted / Store as an integer.

Example:—

```
#include <iostream>
using namespace std;
```

```
void add(int a, int b) {
    int sum = a + b;
```

```
    cout << "sum" << sum << endl; }
```

```
void add(float a, float b) {
    float sum = a + b;
```

```
    cout << "sum" << sum << endl; }
```


int main()

```
add(2.5, 3.5);
```

```
add(2, 3);
```

Compiler - Error & ambiguous problem

⇒ যেহেতু আমরা যদি integer variable চাইলে or floating value হলে 2 or 3 -এর integer notation টি শুধু হবে

- ~~গরম~~ এর ২য় value থেকে 2 আর 3 - ~~নতুন~~ Store

- ଚାନ୍ଦି ଚାନ୍ଦି ଯାହା ଯୁଗ ଯୁଗ - ଚାନ୍ଦି ଚାନ୍ଦି ଯୁଗ
 - ଆମାଧା ଆମାଧା ଆମାଧା ଆମାଧା - ଆମାଧା ଆମାଧା ଯୁଗ

- ଆମାଧା ଆମାଧା ଅଧିକ different Variable ହିମ୍ ।
 - ଏକ-କଥା ସ୍ମାର floating Variable - ଏକ-କଥା ସ୍ମାର ।
 So, Compiler ଏକ-କଥା - ଏକ-କଥା ।

So, compiler will get confused - RCE error -
due to ambiguous problem it shows error.

void access - error
function use - error

Example:-

```
#include <iostream>
using namespace std;
void print(int a) {
    a++;
    cout << "a:" << a << endl;
}
int main() {
    int a = 5;
    print(a);
    cout << "main a:" << a << endl;
}
```

Output:-
a: 6
main a: 5

⇒ Error because of:-

- 1. void function local variable = a address is same as main function local variable = a address same

Correct Error

Global Variable:

⇒ Global Variable হলো এমন একটি Variable যেখানে Variable-টি Store করা হয়, যেখানে থেকে প্রতিবার function এ কল করা যাবে।

Examples

```
#include <iostream>
using namespace std;
```

```
int a = 5; → The global function.
void print() {
    a++;
    cout << "function of a: " << a << endl;
}
```

```
int main() {
    print();
    cout << "main a: " << a << endl;
}
```

Output:
function of a: 6
main a: 6

Because:

⇒ Global Variable এ যেহেতু আমরা update value print
করি, সেহেতু আমরা print() এর Void function-টি
আমরা call করলে সেখানে a-এর value ৬ থেকে বৃদ্ধি পাবে
৭। ২য়, যখন main function এ a=6 হবে।

What if we called Void function Later:-

```
#include <iostream>
```

```
using namespace std;
```

```
int a = 5;
```

→ The global function

```
void print() {
```

```
    a++;
```

```
    cout << "function a: " << a << endl; }
```

```
int main() {
```

```
    cout << "main a: " << a << endl;
```

```
    print();
```

```
}
```

Output:-

main a: 5
function a: 6

Because:-

main

void function is called first so value

is 5. After that, print() is called so value

update value 6.

Call by Value

⇒ In call by value, the function receives a copy of the original value of the argument, which means any modification made to the parameter inside the function does not affect the original value of the variable outside the function.

Q:- Do Swap Operation using call by value

```
#include <iostream>
```

```
using namespace std;
```

```
void Swap (int a, int b) {
```

```
    cout << "Before a:" << a << "Before b:" << b << endl;
```

```
    int temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
    cout << "After a:" << a
```

```
    << "After b:" << b << endl;
```

```
}  
int main() {
```

```
    int x = 5;
```

```
    int y = 10;
```

```
    cout << "Before x:" << x << "Before y:" << y << endl;
```

```
    swap(x, y);
```

```
    cout << "After x:" << x << "After y:" << y << endl;
```


Output—
money

Before No 5

Before 4:10

Before a: 5

Before $b \rightarrow 10$

After $a \pm 10$

Altern $b = 5$

After no 5

After $y = 10$.

⇒ main function থেকে কোন মান Value pass করা
 - void function দ্বারা কোন swap operation হি
 - কিন্তু main function এ কোন বদল
Swap হয় না কোন Value প্রদান কোন নিবর্তন হয় নি.

Call by reference / Address:

⇒ In call by reference, the function receives the actual reference or memory address of the argument, which allows the function to modify the original value of the variable outside the function. Any change made to the parameters inside the function affects the original value of the variable.

Q:- What to do if the swap operation is also done inside the void function:-

⇒ यदि void function में swap operation है तो यदि हम main function में नहीं करते हैं तो हमें हमारे pointer का उपयोग करना होगा।

Some basic of the pointers

& = मान Address को दर्शाता है & का उपयोग करना।

* = यदि pointer initialisation के प्रयोग।

```
#include <iostream>
```

```
using namespace std;
```

```
void swap(int *a, int *b)
```

```
{  
    cout << "Before a" << *a << "Before b" << *b << endl;
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
    cout << "After a" << *a << "After b" << *b << endl;  
}
```



```

int main() {
    int x = 5;
    int y = 10;
    cout << "Before x:" << x << "Before y:" << y << endl;
    swap(x, y);
    cout << "After x:" << x << "After y:" << y << endl;
}

```

Output:

```

Before x: 5    Before y: 10
Before a: 5    Before b: 10
After a: 10    After b: 5
After x: 10    After y: 5

```

#Exercise 02:

#include <iostream>

using namespace std;

void Increment(int &a) {

a = a + 1;

cout << "function a:" << a << endl; }

int main() {

int x = 4;

Increment(&x);

cout << "main x:" << x << endl; }

Output:

```

function a: 5
main x: 5

```


Structures:

☞ A Structure is a collection of variables of different types under a single name.

☞ It allows us to define our custom own custom data type with a set of attributes that can be accessed and manipulated as a single unit.

☞ A Structure is a user define data type.

* A Structure is defined using the "Struct" keyword.

Ex:-

```
struct Person {  
    String name;  
    int age;  
    float salary;  
};
```

⇒ struct is a user define data type.

☞ In this Example, the Structure is called "Person". has three members.

1. A String Variable called "name".

2. An integer Variable called "age".

3. A float Variable called "Salary".

⇒ Now, we can create an instance of a structure by declaring a variable of that type, like this:—

```
Person p1;
```

⇒ Now we can access the members of a structure using the dot operator, like this:—

```
p1.name = "Mohsin";
```

```
p1.age = 21;
```

```
p1.salary = 20000.5;
```

→ Structure can be very useful for organizing data in a program, especially when dealing with complex data structure.

Q: Write a C++ program that defines a Structure "StudentInfo" with the following members:-

- name (string)

- ID (string)

- age (int)

- cgpa (float)

Create two Student Object as its argument and display the details of each Student.

```
#include <iostream>
```

```
using namespace std;
```

```
struct StudentInfo {
```

```
    string name;
```

```
    string ID;
```

```
    int age;
```

```
    float cgpa;
```

```
};
```

```
int main {
```

```
    StudentInfo s1;
```

```
    s1.name = "Swarcwar";
```

```
    s1.ID = "23-50100-1";
```

```
    s1.age = 23;
```

```
    s1.cgpa = 3.79;
```



```

cout << "Student-1 Name:" << s1.name << endl;
cout << "Student-1 ID:" << s1.ID << endl;
cout << "Student-1 CGPA:" << s1.cgpa << endl;
cout << "Student-1 Age:" << s1.age << endl;
cout << "-----" << endl;

```

Student info: s2;

```

s2.name = "Rahat";
s2.ID = "23-50231-1";
s2.cgpa = 3.74;
s2.age = 22;

```

```

cout << "Student-2 Name:" << s2.name << endl;
cout << "Student-2 ID:" << s2.ID << endl;
cout << "Student-2 CGPA:" << s2.cgpa << endl;
cout << "Student-2 Age:" << s2.age << endl;

```

Output:

```

Student-1 Name: Swarcwor
Student-1 ID: 23-50109-1
Student-1 CGPA: 3.70
Student-1 Age: 23
-----

```

Student-2 Name: Rahat

Student-2 ID: 23-50231-1

Student-2 CGPA: 3.74

Student-2 Age: 22

Q: Now Create a function named "probation (float cg)" which will take the cgpa from the main function. Afterwards this function checks, if the cgpa is greater than 2.50, the student will be not a probation student, otherwise will be a probation student.

```
#include <iostream>
using namespace std;
struct studentInfo {
```

```
    string name;
    string ID;
    int age;
    float cgpa; }
```

```
void probation (float cg) {
    if (cg < 2.50) {
        cout << "probation student" << endl;
    } else { cout << "Not in probation" << endl; }
}
```

```
int main () {
```

```
    studentInfo s1 = { "Swarwar", "23-5012-1", 22, 2.40 };
    cout << "Student - 1 Name:" << s1.name << endl;
    cout << "Student - 1 ID:" << s1.ID << endl;
    cout << "Student - 1 Age:" << s1.age << endl;
    cout << "Student - 1 CGPA:" << s1.cgpa << endl;
    Probation (s1.cgpa);
}
```



```
cout << "-----" << endl;
```

```
StudentInfo s2 = { "Sami", "23-50205-1", 23, 3.80 };
```

```
cout << "Student-2 Name:" << s2.name << endl;
```

```
cout << "Student-2 ID:" << s2.ID << endl;
```

```
cout << "Student-2 Age:" << s2.age << endl;
```

```
cout << "Student-2 CGPA:" << s2.CGPA << endl;
```

```
Probation (s2.CGPA);
```

Output:

Student-1 Name: Swarwaro

Student-1 ID: 23-50102-1

Student-1 Age: 22

Student-1 CGPA: 2.40

Probation Student

Student-2 Name: Sami

Student-2 ID: 23-50205-1

Student-2 Age: 23

Student-2 CGPA: 3.80

Not in probation

Q:- What if user wants to input the value:-

```
#include <iostream>
```

```
using namespace std;
```

```
struct StudentInfo {
```

```
    string Name;
```

```
    string ID;
```

```
    int age;
```

```
    float CGPA; };
```

```
int main () {
```

```
    StudentInfo s1;
```

```
    cout << "Enter Your Name:";
```

```
    cin >> s1.Name;
```

```
    cout << "Enter Your ID:";
```

```
    cin >> s1.ID;
```

```
    cout << "Enter Your age:";
```

```
    cin >> s1.age;
```

```
    cout << "Enter Your CGPA:";
```

```
    cin >> s1.CGPA;
```

```
    cout << "Name:" << s1.Name << endl;
```

```
    cout << "ID:" << s1.ID << endl;
```

```
    cout << "Age:" << s1.age << endl;
```

```
    cout << "CGPA:" << s1.CGPA << endl;
```

```
}
```


Output

Enter your Name: Alif Hasan Khan

Enter your ID: 23-50808-1

Enter your age: 22

Enter your GPA: 3.70

Name: Alif

ID: 23-50808-1

Age: 22

GPA: 3.70

Observation

→ লক্ষ্য করুন যে আমরা Name Input দিয়েছি Alif Hasan Khan
যেহেতু এখানে Compiler আমাদের Output দিয়েছে only Alif
এর কারণ হল যে আমরা user থেকে Value input করে
আমরা যদি কোনো স্পেস (space) দিই তবে Compiler
space টিকে termination হিসেবে মান করে। তাই এখানে
only Alif এই position-তে Print করে।

Getlines

So, no problem to overcome - এটা হল আমরা

"getline" ব্যবহার করি।

```
int main()
```

```
{ StudentInfo s1;
```

```
cout << "Enter your name:";
```

```
getline(cin, s1.name);
```

```
cout << "Name:" << s1.name << endl;
```

Output
from

Enter your Name: Alif Hasan Khan
Name: Alif Hasan Khan.

#OOP - Object-Oriented Programming

Access modifier or Access Specifier

⇒ An Access modifier or Access specifier is a keyword in oop languages, that determines the accessibility of class members from other parts of the program.

There are three Access modifiers in ++C-

① Public: Member declared as public can be accessed from anywhere in the program, including outside the class.

② Private: Member declared as private can be accessed within the class itself. This means that other parts of the program can't access or modify private data members directly.

③ Protected: Member declared as protected can be accessed from within the class and its subclasses. It's similar to private, but they can be inherited by subclasses.

Q:- Create a c++ class named "StudentInfo" that stores information about a student's name, ID, age and cgpa. Declare four public data member in the class to represent these attributes. Define a public member function named "PrintStudentInfo()" that print the information to the Console. Write a program that creates two StudentInfo object, initializes their value with the following attributes, and calls the "PrintStudentInfo" member function to print their information to the Console.

```
#include <iostream>
using namespace std;
class StudentInfo
{
public: // Access modifier or specifier.
    string studentName;
    string studentID;
    int age;
    float cgpa;
    void PrintStudentInfo()
    {
        cout << "Student Name:" << studentName << endl;
        cout << "Student ID:" << studentID << endl;
        cout << "Student Age:" << age << endl;
        cout << "Student CGPA:" << cgpa << endl;
    }
};
```



```

int main ()
{
    StudentInfo s1 = { "Jam", "23-50187-1", 21, 3.90 };
    s1.PrintStudentInfo();

    StudentInfo s2 = { "Ayon", "23-50215-1", 22, 3.85 };
    s2.PrintStudentInfo();
}

```

Output—

Student Name: Jam
 Student ID : 23-50187-1
 Student Age: 21
 Student GPA: 3.90
 Student Name: Ayon
 Student ID : 23-50215-1
 Student Age: 22
 Student GPA: 3.85

Rules—

1. Public Access modifier declare করা হয় এর নিচে বা
 থাকবে সব public. Until অন্য modifier declare করা
 যায়.
2. class এর ভিতরে যখন আমরা function চাইতে চাই.
 তখন সে function টিকে method বলা হয়.

৩. যদি class-এর মতী কোনো Access modifier না দেওয়া
হয় তাহলে তুমি by default private ব্যবহার্য থাকে।
৫. public ব্যবহারে কোন বাস class-এর বাইরে থেকে
access করা যায়।

৬. Private and Protected-এর ক্ষেত্রে class-এর বাইরে
থেকে access করা যায় না। কোন বাস যে class-এর মতী থেকে কোনো

Q:- Create a c++ class named "StudentInfo" that stores
Information about a student's name, ID, age, and CGPA.
In the class Student name and ID is a public Data
member. Student age is a protected Data member. Student
CGPA is a private data member. Define a public member
function named "SetInfo()" that set student age and CGPA.
Also Define another function named "PrintStudentInfo()" that
print student information to the console:-

Use of Public, Private and Protected together:-

```
#include <iostream>
using namespace std;
class StudentInfo
{
public:
    String StudentName;
    String StudentID;
```


Protected:

int age;

Private:

float gpa;

public:

```
void SetInfo (int aq, float eq) {  
    age = aq;  
    gpa = eq;  
}
```

```
void PrintStudentInfo () {
```

```
    cout << "Student Name:" << StudentName << endl;
```

```
    cout << "Student ID:" << StudentID << endl;
```

```
    cout << "Student Age:" << age << endl;
```

```
    cout << "CGPA:" << gpa << endl;
```

```
}
```

```
int main () {
```

```
    StudentInfo s1;
```

```
    s1.StudentName = "Nishat";
```

```
    s1.StudentID = "23-50210-1";
```

```
    s1.SetInfo (23, 3.73);
```

```
    s1.PrintStudentInfo ();
```

Output:-

Student Name : Nishat

Student ID : 23-50210-1

Student Age : 23

CGPA : 3.73

⇒ যদি কোনো private type data to value input
 চাই তবে সে class-র বাইরে একটি public type method
 নিয়ে. এই method-র বাইরে same type-র variable নিয়ে
 একটা variable দিয়ে private type data স্বল্পে স্বল্পে
 align করে নিতে হবে।

```
private:
    int age;
    float cgpa;
public:
    void setInfo (int aq, float eq) {
        age = aq;
        cgpa = eq;
    }
```

-এখানে main function-এর public type-র method-র
 value align করে. আর private-র বাইরে value
 align করে হবে।

⇒ যদি কোন protected type data to value input নিতে
 চাই:

As same as private.

Constructors and Destructors

- ⇒ এটি method এর মত মিসিংস, class এর মত method এর বিরুদ্ধে এটি প্রচার করা হয়।
- ⇒ একটি Constructor এর নাম এর class এর নাম same থাকে।
- ⇒ Constructor এর ফোর Data type নেই।
- ⇒ Constructor Parameterized হতে পারে।
- ⇒ ফোর Constructor এর মধ্যে Parameters না থাকলে তাতে default Constructor বলা হয়।
- ⇒ Destructor as same as Constructor.
- ⇒ Destructor এর নাম Always $\sim A()$ দিয়ে শুরু করা হয়।
- ⇒ Destructor কখনো Parameterized হতে পারে না।
- ⇒ প্রত্যেক Object Creation এর সাথে সাথে Constructor call হবে।
- ⇒ Constructor call হয় ডেফল্ট থেকে দিচ্ছে।
- ⇒ Destructor call হয় নিচ থেকে ডেফল্ট দিচ্ছে।

Ex-01

```
#include <iostream>
using namespace std;
class A {
public:
    A() { cout << "Default Constructor Called" << endl; }
    ~A() { cout << "Destructor called" << endl; }
};

int main() {
    A a1;
    A a2;
}
```

Output
from

Default Constructor Called
Default Constructor called
Destructor called
Destructor called.

Ex802

```
#include <iostream>
using namespace std;
```

```
class A {
public:
```

```
    A() { cout << "Default constructor called" << endl; }
```

```
    A(int a) { cout << "Parameterized constructor" << endl; }
```

```
    void methodA() {
```

```
        cout << "Method called" << endl; }
```

```
    ~A() { cout << "Destructor called" << endl; }
};
```

```
int main() {
```

```
    A a1;
```

```
    a1.methodA();
```

```
    A a2(2); }
```

Output

Default constructor called

Method called

Parameterized constructor called

Destructor called

Destructor called.

ex:- Paramterized Constructor $\rightarrow$ Any object
Creation $\rightarrow$ Not Not Value Pass $\rightarrow$ No func 20.

Exo 3

```
#include <iostream>
using namespace std;
```

```
class student{
    string name;
    string ID;
    int age;
    float cpga;
```

Publics

```
Student(String nm, String id){
    name = nm;
```

```
    ID = id; }
```

```
void setInfo(int ag, float cg){
```

```
    age = ag;
```

```
    cpga = cg; }
```

```
- Student() {
```

```
    cout << "Student Name:" << name << endl;
```

```
    cout << "Student ID:" << ID << endl;
```

```
    cout << "Student Age:" << age << endl;
```

```
    cout << "CGPA" << cpga << endl; }
```

```
};
```



```
int main()
```

```
Student s1("Ananno", "23-50108-1");
```

```
s1.setInfo(24, 3.75);
```

```
Student s2("Nihan", "23-50106-1");
```

```
s2.setInfo(23, 3.73);
```

Output
from

Student Name: Nihan

Student ID: 23-50106-1

Student Age: 23

CGPA : 3.73

Student Name: Ananno

Student ID: 23-50108-1

Student Age: 24

CGPA : 3.75

Inheritance

⇒ Inheritance is a fundamental concept in oop that allows a class to inherit properties from another class.

There are different type of inheritance.

1. Single inheritance
2. Multiple inheritance
3. Multilevel inheritance
4. Hybrid inheritance
5. Hierarchical inheritance

① Single inheritance:-

⇒ Where a class inherits from only one base class.

② Multiple inheritance:-

⇒ Where a class inherits from two or more base class.

③ Multilevel Inheritance:-

⇒ Where a derived class is used a base class for another derived class.

④. Hybrid Inheritance:

⇒ It's a combination of multiple inheritance and hierarchical inheritance. Where one or more base classes have multiple derived classes.

⑤. Hierarchical Inheritance:

⇒ Where multiple derived classes are created from a single base class.

⇒ Inheritance is specified using the colon (:) Symbol. After the class name, followed by the access specifier and the name of base class.

Example 9.1 for Single inheritance:

```
#include <iostream>
using namespace std;
```

```
class person {
```

```
    String Name;
```

```
    int age;
```

```
    int NID;
```

```
public:
```

```
    void setPersonInfo (String nm, int ag, int
```

```
        Name = nm;
```

```
        age = ag;
```

```
        NID = NI;
```

```
    }
```



```
Void PersonInfoPrint()
```

```
    Cout << " Name:" << Name << endl;
```

```
    Cout << " NID:" << NID << endl;
```

```
    Cout << " Age:" << age << endl; }
```

```
};
```

child class/sub class

Parent class/ Super class.

```
class Student : public Person {
```

String
float

```
    Student ID;
```

```
    CGPA;
```

public:

```
Void Set Student Info (String id, float cg)
```

```
    StudentID = id;
```

```
    CGPA = cg; }
```

```
Void StudentInfoPrint()
```

```
    Cout << " Student ID:" << StudentID << endl;
```

```
    Cout << " CGPA:" << CGPA << endl; }
```

```
};
```

```
int main()
```

Student obj;

```
obj.SetPersonInfo("ELI", 22, 675020);
```

```
obj.SetStudentInfo("23-50104-1", 3.22);
```

```
obj.PersonInfoPrint();
```

```
obj.StudentInfoPrint();
```

```
{
```


Output:

Name: ELI
NID: 675020
Age: 22
Student ID: 22-50104-1
CGPA: 8.92

Example 2:

```
#include <iostream>  
using namespace std;
```

```
class A {
```

```
public:
```

```
A() {
```

```
    cout << "Default constructor of A" << endl; }
```

```
A(int a) { cout << "Parameterized constructor of A" << endl; }
```

```
~A() { cout << "Destructor of A" << endl; }
```

```
};  
  
class B: public A {
```

```
public:
```

```
B() { cout << "Default constructor of B" << endl; }
```

```
B(int a) { cout << "Parameterized constructor of B" << endl; }
```

```
~B() { cout << "Destructor of B" << endl; }
```

```
};
```



```

int main()
{
    B b1;
    A a1(2);
    B b2(2);
}

```

Output

Default Constructor of A.
 Default Constructor of B.
 Parameterized Constructor of A.
 Default Constructor of A.
 Parameterized Constructor of B.
 Destructor of B
 Destructor of A
 Destructor of A
 Destructor of B
 Destructor of A

Observation

⇒ ~~When we use~~ Parameterized Constructor of B, Value
 Pass - ~~When we use~~ child class - ~~When we use~~ Parameterized Constructor
 call - ~~When we use~~ Parent class - ~~When we use~~ Default Constructor to
 call - 2nd. (When call - 2nd time - ~~When we use~~ Parent class - ~~When we use~~
 Parameterized Constructor to.

- ~~As~~ problem to Overcome - ~~When we use~~ ~~When we use~~ Parent
 class - ~~When we use~~ Parameterized Constructor call - ~~When we use~~ 1st time
~~When we use~~ Parameterized Constructor of B - ~~When we use~~ Value Pass - ~~When we use~~
~~When we use~~ Parameterized Constructor of A - ~~When we use~~ Value Inherit

One Pass - one call - one parameterized
Constructor of A call - one

Like:-

```
B(int a) : A(a) {}
```

Example 3

```
#include <iostream>  
using namespace std;
```

```
class A {
```

```
public:
```

```
A() { cout << "Default Constructor of A" << endl; }
```

```
A(int a) { cout << "Parameterized Constructor of A" << endl; }
```

```
~A() { cout << "Destructor of A" << endl; }
```

```
};
```

```
class B : public A {
```

```
public:
```

```
B() { cout << "Default Constructor of B" << endl; }
```

```
B(int a) : A(a) { cout << "Parameterized Constructor of B" << endl; }
```

```
~B() { cout << "Destructor of B" << endl; }
```

```
};
```



```
int main() {
```

```
    B b1;
```

```
    A b2;
```

```
    B a2(2);
```

```
}
```

Input: —

Output: —

Default Constructor of A

Default Constructor of B

Default Constructor of A

Parameterized Constructor of A

Parameterized Constructor of B

Destructor of B

Destructor of A

Destructor of A

Destructor of B

Destructor of A

Static Variable:—

⇒ Static Value is call by reference — এর মত কাজ করে
that mean's update value চুক্তি।

Example 1

```
#include <iostream>
using namespace std;
Void Count (int a) {
    static int c = a;
    c++;
    cout << "c:" << c << endl; }
int main() {
    int b = 4;
    Count (b);
    Count (b); }
```

Output:—

c: 5
c: 6

Example 2

```
#include <iostream>
using namespace std;
Void Count (int a) {
    static int c = a;
    c = c + 2;
    cout << "c:" << c << endl; }
```



```

int main() {
    int a = 4;
    Count(a);
    Count(a);
    Count(a);
}

```

Output:-

c: 6
 c: 8
 c: 10

Static Variable in class:-

```

#include <iostream>
using namespace std;

```

```

class A {

```

```

    public:

```

```

        int a = 2;

```

```

        void methodA() {

```

```

            a++;

```

```

            cout << "a:" << a << endl;
        }

```

```

int main() {

```

```

    A a1;

```

```

    a1.methodA();

```

```

    A a2;

```

```

    a2.methodA();
}

```

Output:-

a: 3
 a: 3

Observation:-

⇒ ସ୍ଟାଟିକ Static Variable କେବଳ ଏକ ଥର ସୂଚକ
ସ୍ଟାଟିକ ଆବଶ୍ୟକ କେବଳ Update Value କରି
ପାରି.

Examples of

⇒ Static Variable ର Value class ର ସହିତ Assign କରି
ପାରି ନା. ଏକ Value Assign କରି ଶୁ initialise
କରି ଶୁ class ର ସହିତ main function ର
ସହିତ.

Like:-

name of the class name of the static variable
`int A::a = 2;`
↓
Assign Value of the static variable.

```
#include <iostream>
using namespace std;
```

```
class A{
```

public

```
static int a;
```

```
void methodA(){
```

```
    a++;
```

```
    a = a + 2;
```

```
    cout << "a:" << a << endl;
```

```
};
```


int A88 a=2; // Static Variable initialization

int main() {

A a1;

a1.methodA();

A a2;

a2.methodA();

}

Output: —
mom

a8 5

a8 8